1. System Architecture & Tech Stack

The project is structured using the Flask 'Blueprint' pattern. This means the backend isn't just one huge file, but separated into logical modules like traffic.py, waste.py, complaints.py, etc. This is a crucial point for the Viva: it shows you understand modular architecture and separation of concerns.

Tech Stack Details:
- Backend: Python, Flask (lightweight, perfect for fast hackathons).
- Database: SQLite via SQLAlchemy ORM (Object Relational Mapping). We used SQLite because it requires zero configuration, which is ideal for hackathon demos. For production, you'd mention migrating to PostgreSQL.
- Frontend: Vanilla JS, HTML, CSS. Leaflet.js used for rendering the open-source map.
- Real-time Data: We used APScheduler to run a background Python job that constantly injects simulated IoT sensor data into the database.

2. Machine Learning Models (Crucial for Judge Q&A)

A. Traffic Congestion Prediction

- Algorithm: RandomForestRegressor (Scikit-Learn).
- Inputs/Features: Vehicle count, average speed, current congestion level.
- Target: Predicts the congestion level 30 minutes into the future.
- Why Random Forest?: Traffic isn't strictly linear. Speed drops rapidly once a road hits a certain capacity. Trees capture these non-linear thresholds better than simple linear models.
- Fallback Mechanism: If the DB has < 30 rows of data (like at the start of the app), the model falls back to a clever heuristic (mean reversion) rather than crashing.

B. Waste Bin Fill Prediction

- Algorithm: LinearRegression (Scikit-Learn).
- Inputs/Features: Time (index of the last 24 readings).
- Target: Predicts fill percentage 5 hours ahead.
- Why Linear?: Trash accumulation over a few hours is generally a steady, linear trend. Overcomplicating it with deep learning would lead to overfitting.

3. Frontend Mechanics & The JS Refactor

Initially, the frontend JavaScript was a massive monolith (app.js). We refactored it into feature-specific files (traffic.js, waste.js, environment.js, etc.).

How data updates: The frontend uses `setInterval` to 'poll' the backend /api/ endpoints every 6 seconds using the Fetch API. We use `Promise.all` to fetch multiple endpoints concurrently, which speeds up the dashboard loading time significantly.

4. Potential Viva / Judge Questions & Answers

Q: How does your system get 'Live' data?

A: In a real city, we would have IoT sensors (cameras, bin weight sensors). For this hackathon, we built a custom Python Simulator using APScheduler that generates realistic telemetry and writes it to our database every few seconds.

Q: Why did you use HTTP Polling instead of WebSockets?

A: HTTP polling (fetching data every X seconds) was chosen for speed of development and simplicity during the hackathon. It works well for dashboards. If asked how to scale it, say: 'In production, we would upgrade to WebSockets (e.g., Socket.IO) to push data from server to client instantly, reducing HTTP overhead.'

Q: What if the AI model receives bad data?

A: We implemented boundaries and fallbacks. For example, the traffic model checks if there are enough rows to train. The prediction outputs are clamped between 0% and 100% using max/min functions so the UI never breaks.

5. Ultimate Hackathon Learnings (For the Future)

- Modularization Saves Lives: Lumping all code into app.py or app.js leads to nightmare merge conflicts and bugs. Segregating code early into 'features' makes debugging 10x faster.

- Handle Nulls Gracefully: Always wrap DOM element selections (document.getElementById) in an 'if' check. If an element is missing on a specific HTML page, an unhandled null exception will crash the entire script.

- AI is more than just API Calls: Training simple Scikit-Learn models on-the-fly gives you complete control and is much faster/cheaper than hitting external LLMs for numerical forecasting.

- UI/UX Matters: Judges love dark mode, clean CSS variables, and interactive elements (like the Leaflet map tooltips). It makes the project feel finished.